

INFORME DE HALLAZGOS - SESIÓN 8 AUTÓNOMA

Dependencias entre Categorías y Productos - PharmaSoft

Estudiante: Pari

B1. Resumen de la ejecución

Grupo	Casos	Pasan	Fallan
Altas	7	6	1
Cambios	6	3	3
Bajas	5	3	2
TOTAL	18	12	6

Se ejecutaron 18 casos: 14 base y 4 propios. Se obtuvieron 12 resultados conformes y 6 fallas. Las fallas se concentran en reglas de dependencia que la interfaz intenta proteger, pero que el backend no valida de forma consistente.

B2. Fichas por hallazgo

H-01 - Caso A-04

Descripción	La API permitió registrar un producto en la categoría Dermocosmética inactiva y devolvió 201 Created. La regla esperaba rechazo 409.
Severidad	Alta - puede generar datos inconsistentes.
Evidencia	A-04_postman.png; POST /api/v1/productos con categoriald=7.
Causa probable	Backend: método de creación/registro de productos no comprueba que la categoría encontrada esté activa antes de persistir.
Corrección propuesta	En la capa de negocio del backend, después de obtener la categoría, validar estado=true. Si está inactiva, lanzar conflicto 409 antes de guardar.

H-02 - Caso C-02

Descripción	La SPA no ofrece categorías inactivas, pero un PUT directo permitió mover el producto a Dermocosmética inactiva y devolvió 200 OK.
Severidad	Alta - la API puede romper la dependencia aunque la SPA la oculte.
Evidencia	C-02_spa.png y C-02_postman.png; PUT /api/v1/productos/15.
Causa probable	Backend: método de actualización de producto valida existencia de categoría, pero no su estado activo.
Corrección propuesta	Aplicar en actualización la misma validación de categoría activa propuesta para creación y responder 409 si estado=false.

B2. Fichas por hallazgo - continuación

H-03 - Caso C-03

Descripción	Se desactivó Cuidado Personal aun cuando tenía un producto activo. El backend respondió 200 OK.
Severidad	Alta - deja productos activos asociados a una categoría inactiva.
Evidencia	C-03_spa.png; PUT /api/v1/categorias/8.
Causa probable	Backend: actualización de categoría no verifica si existen productos activos antes de pasar estado a false.
Corrección propuesta	Regla elegida: impedir la desactivación mientras existan productos activos. Consultar ProductoRepository por categoriaId y estado=true; si existen, responder 409.

H-04 - Caso C-04

Descripción	Un formulario quedó abierto con Temporal S8 activa; en otra pestaña se desactivó la categoría y luego el registro original terminó con 201 Created.
Severidad	Alta - falla de concurrencia y validación tardía.
Evidencia	C-04_spa.png; POST /api/v1/productos después de desactivar la categoría.
Causa probable	La SPA valida con datos cargados antes, pero la única capa con estado actualizado al confirmar es el backend. La API no revalida categoría activa al guardar.
Corrección propuesta	Revalidar siempre en backend justo antes de persistir. La SPA puede seguir ocultando inactivas, pero no debe ser la autoridad final.

H-05 - Caso B-02

Descripción	Al intentar dar de baja por segunda vez el producto 13, la API devolvió 404 en lugar de 409 'ya se encuentra inactivo'.
Severidad	Media - el dato puede seguir protegido, pero el contrato HTTP y el mensaje son incorrectos.
Evidencia	B-02_postman.png; DELETE /api/v1/productos/13.
Causa probable	Backend: la búsqueda usada por el método de baja probablemente excluye registros inactivos, por lo que no distingue 'inactivo' de 'inexistente'.
Corrección propuesta	Buscar el producto sin filtrar por estado. Si existe y estado=false, responder 409; si realmente no existe, responder 404.

H-06 - Caso B-04

Descripción	La categoría Cuidado Personal fue eliminada con 204 aun cuando tenía un producto dado de baja asociado.
Severidad	Alta - puede perder la referencia histórica usada por reportes o ventas.
Evidencia	B-04_spa.png; DELETE /api/v1/categorias/8.
Causa probable	Backend: la regla de eliminación parece considerar solo productos activos o no considerar productos dados de baja.
Corrección propuesta	Para conservar trazabilidad, contar todos los productos asociados sin filtrar estado. Si existe alguno, impedir la eliminación y responder 409.

B3. Cobertura de reglas

Regla	SPA	API	Evaluación actual
Existencia de categoría	Parcial	Sí	La API devuelve 404 para categoríaId inexistente.
Categoría activa	Sí	No	La SPA oculta/bloquea, pero A-04 y C-02 pasan por API.
Nombre único	No preventiva	Sí	API devuelve 409 al crear o actualizar duplicados.
No eliminar categoría con productos	No	Parcial	Funciona con productos activos, pero B-04 mostró eliminación con producto inactivo.
Baja lógica	Sí en presentación	Parcial	Primera baja 204; segunda baja devuelve 404 en lugar de 409.

La principal brecha es la regla 'categoría activa': la SPA sí la aplica al select y al formulario, pero la API no la impone en las operaciones de creación y actualización. Por ello, un cliente distinto de la SPA puede insertar o mover productos a categorías inactivas.

B4. Recomendación sobre el filtro por categoría

Actualmente ProductoList obtiene una página del backend y después aplica el filtro por `categoriaId` sobre el contenido ya recibido. Esto significa que el filtro solo conoce los registros de la página actual. La mejora 'Ver productos' mitiga el problema cargando hasta 100 registros, pero la solución completa debe estar en el backend.

Cambio propuesto en el endpoint

Agregar el parámetro opcional `categoriaId` a GET `/api/v1/productos`. Ejemplo:
`/api/v1/productos?pagina=0&tamano;=10&ordenarPor;=nombre&direccion;=asc&categoriaId;=6`. Cuando el parámetro exista, la paginación debe ejecutarse sobre los productos de esa categoría, no sobre todos.

Cambio propuesto en Spring Data

En `ProductoRepository`, incorporar un método paginado como `findByCategoriaId(Long categoriaId, Pageable pageable)`, o una `Specification` equivalente si se combinan más filtros. De este modo, el `totalElementos` y `totalPaginas` corresponden al subconjunto filtrado.

Cambio propuesto en ProductoService del frontend

Extender `listar()` para recibir `categoriaId` opcional y agregarlo a `HttpParams` cuando tenga valor. `ProductoList` dejaría de ejecutar `lista.filter(...)` localmente y pediría al servidor exactamente la categoría seleccionada. El query param recibido desde 'Ver productos' seguiría alimentando `categoriaFiltro`, pero ahora también viajaría a la API.

Mejora implementada en la SPA

Se agregó el botón 'Ver productos' en Categorías, que navega a `/productos?categoriaId=`. `ProductoList` recibe `categoriaId` como input, selecciona la categoría, carga tamaño 100 y muestra la nota de contexto. La evidencia final muestra `?categoriaId=6`, selector Vitaminas y la tabla filtrada.

B5. Preguntas de análisis

1. ¿Por qué no basta con que la SPA oculte las categorías inactivas?

Porque la SPA es solo un cliente. A-04 y C-02 demostraron que Postman puede llamar directamente a la API y saltar la restricción visual. El formulario de productos sí evita seleccionar una categoría inactiva, pero la integridad debe imponerse también en la capa de negocio del backend antes de persistir.

2. ¿Qué debería pasar al desactivar una categoría con productos activos?

Elegimos impedir la desactivación mientras existan productos activos. Es la opción más segura para ventas, porque mantiene coherencia entre catálogo y operaciones: un producto disponible no debería quedar ligado a una categoría inactiva. La API debería responder 409 y exigir primero reubicar o dar de baja esos productos.

3. En B-04, ¿deberían contarse productos dados de baja?

Sí. Aunque el producto esté inactivo, su relación puede ser necesaria para auditoría, reportes y ventas históricas. Si se elimina físicamente la categoría, un reporte antiguo podría perder el nombre o la clasificación original. Por eso proponemos impedir la eliminación mientras exista cualquier producto asociado.

4. ¿Qué revela C-04 sobre el momento de validación?

Revela que validar solo al cargar el formulario no es suficiente. El estado puede cambiar entre la lectura y el guardado. La única capa que puede resolver correctamente esa condición en el momento de la escritura es el backend, porque consulta el estado vigente dentro de la operación que persiste el producto.

5. Si el backend corrige A-04, ¿qué cambia en la SPA?

La SPA no necesita cambiar su comportamiento principal: debe seguir mostrando solo categorías activas y comunicando errores del servidor. Lo que cambia es que, incluso si alguien evita la interfaz o trabaja con datos desactualizados, el backend responderá 409. La validación visual sigue siendo útil para experiencia de usuario, pero deja de ser la única defensa.

Conclusión

Las pruebas muestran que la SPA cubre varias validaciones de experiencia de usuario, pero la integridad entre Categorías y Productos todavía depende de validaciones faltantes o incompletas en la API. Los hallazgos A-04, C-02 y C-04 son los más críticos porque permiten persistir productos en categorías inactivas. La recomendación principal es centralizar las reglas de dependencia en la capa de negocio del backend y mantener la SPA como validación complementaria.